Scanner Development Document

Catalogue

1. Obtain the bar code

By the way of code calls, generally we commonly use the output mode is: broadcast mode, broadcast mode needs to first through the SDK API to set the PDA output mode to broadcast mode, the specific operations are as follows

1.1 Set output mode to [Broadcast mode]

```
ScanManager mScanManager = new ScanManager();
mScanManager.switchOutputMode(0);//0:  Broadcast mode 1: Keyboard mode
```

1.2 Registered broadcast

```
IntentFilter filter = new IntentFilter();
filter.addAction(ScanManager.ACTION_DECODE);
registerReceiver(mReceiver, filter);
```

1.3 Listening broadcast callback

```
private BroadcastReceiver mReceiver = new BroadcastReceiver() {
   @Override
   public void onReceive(Context context, Intent intent) {
   String barcodeStr = intent.getStringExtra(ScanManager.BARCODE_STRING_TAG
);//barcodeStr   Get scanned barcode data
   }
};
```

1.4 Unregister (you will not receive broadcast results)

```
unregisterReceiver(mReceiver);
```

2. API

2.1 switchOutputMode【Set output mode】

Definition	public boolean switchOutputMode(int mode);		
Description	Set the barcode output mode after PDA scans the barcode.		
Parameters	name	type	note
	mode	int	0: Broadcast mode (Intent output) 1: Keyboard mode
Return (boolean)	true : Successful false: Failure		
Reference code	mScanManager.switchOutputMode(0);		

2.2 getOutputMode【Get output mode】

Definition	public int getOutputMode();		
Description	Gets the output method of scanning results currently used by PDA		
Parameters	name	type	note
Return (int)	0: Broadcast mode (Intent output) 1: Keyboard mode		
Reference code	mScanManager.getOutputMode();		

2.3 startDecode【Start scanning】

Definition	public boolean startDecode();		
Description	Code set scanning head scanning code (same effect as left and right scanning code keys, if you use keys to trigger scanning, you can not call this method)		
Parameters	name	type	note
Return (boolean)	true : Successful false: Failure		
Reference	mScanManager.startDecode();		

code	

2.4 stopDecode【Stop scanning】

Definition	public boolean stopDecode();		
Description	Code control stops scanning.		
Parameters	name	type	note
Return (boolean)	true : Successful false: Failure		
Reference code	mScanManager.stopDecode();		

2.5 setTriggerMode【Set the trigger scanning mode】

Definition	public void setTriggerMode(TriggeringtgMode);		
Description			
Parameters	name	type	note
	tgMode	Triggering	Triggering.PULSE: Short press triggers decoding or timeout stop Triggering.CONTINUOUS: continuous scan mode Triggering.HOST: Long-press to trigger and release to stop
Return (boolean)	true : Successful false: Failure		
Reference code	mScanManager.setTriggerMode(Triggering.PULSE);		

2.6 getTriggerMode【Gets the trigger scan mode】

Definition	**public Triggering getTriggerMode();**		
Description			
Parameters	**name**	**type**	**note**
Return (Triggering)	**Triggering.PULSE: Short press triggers decoding or timeout stop** **Triggering.CONTINUOUS: continuous scan mode** **Triggering.HOST: Long-press to trigger and release to stop**		
Reference code	**mScanManager.getTriggerMode();**		

2.7 lockTriggler【Disable Scan】

Definition	**public boolean lockTriggler();**		
Description	**Set scan trigger inactivity (disable scan button and scan method).**		
Parameters	**name**	**type**	**note**
Return (boolean)	**true : Successful** **false: Failure**		
Reference code	**mScanManager.lockTriggler();**		

2.8 unlockTriggler【Enable Scan】

Definition	**public boolean unlockTriggler();**		
Description	**Set scan trigger activation (enable scan button).**		
Parameters	**name**	**type**	**note**

Return (boolean)	**true : Successful** **false: Failure**		
Reference code	**mScanManager.unlockTriggler();**		

2.9 openScanner【Open scan engine】

Definition	**public boolean openScanner();**		
Description	**Open the scan engine (generally, it is enabled by default, so you do not need to call this method).**		
Parameters	**name**	**type**	**note**
Return (boolean)	**true : Successful** **false: Failure**		
Reference code	**mScanManager.openScanner();**		

2.10 closeScanner【Turn off scan engine】

Definition	**public boolean closeScanner();**		
Description	**Stop the scan engine. After the scan engine is stopped, the scan cannot be performed**		
Parameters	**name**	**type**	**note**
Return (boolean)	**true : Successful** **false: Failure**		
Reference code	**mScanManager.closeScanner();**		

2.11 setParameterString【Set scanning Parameters (the value is String)】

Definition	public boolean setParameterString(int[] index, String[] value);		
Description	Set scanning Parameters		
Parameters	name	type	note
	index	int[]	Set the Parameterskey corresponding to Parameters. The length becomes longer. You can pass in an arbitrary length of Parameters array.
	value	String[]	Set the corresponding value of the corresponding Parameters, the length should be corresponding to the length of the above key.
Return (boolean)	true : Successful false: Failure		
Reference code	mScanManager..setParameterString(new int[]{PropertyID.WEDGE_INTENT_ACTION_NAME,PropertyID.WEDGE_INTENT_DATA_STRING_TAG},new String[]{"com.pkuhit.intent.action.SCANRECVR","The KeyString you received"});		

2.12 getParameterString【Get scan Parameters (String type)】

Definition	public String[] getParameterString(int[] index);		
Description	Get scan Parameters		
Parameters	name	type	note
Return (String[])	The subscript of the String[] result corresponds to the value of the subscript of Parameters passed in.		
Reference	int[] keys = new		

code	int[]{PropertyID.WEDGE_INTENT_ACTION_NAME,PropertyID.WEDGE_INTENT_DATA_STRING_TAG}; String[] ret = mScanManager.getParameterString(keys);

2.13 setPropertyInts【Set scanning Parameters (value is int type)】

Definition	public boolean setPropertyInts(int[] index, int[] value);		
Description	设置扫描 Parameters		
Parameters	name	type	note
	index	int[]	Set the Parameterskey corresponding to Parameters. The length becomes longer. You can pass in an arbitrary length of Parameters array.
	value	int[]	Set the corresponding value of the corresponding Parameters, the length should be corresponding to the length of the above key.
Return (boolean)	true : Successful false: Failure		
Reference code	mScanManager..setPropertyInts(new int[]{PropertyID.SEND_GOOD_READ_BEEP_ENABLE,PropertyID.SEND_GOOD_READ_VIBRATE_ENABLE},new int[]{2,1});////Turn on sound and vibration in broadcast mode		

2.14 getPropertyInts【Get scan Parameters (int type)】

Definition	public int[] getPropertyInts(int[] index);
Description	Get scan Parameters

Parameters	name	type	note
Return (int[])	The subscript of the result of int[] corresponds to the value of the subscript of Parameters passed in.		
Reference code	int[] keys = new int[]{PropertyID.SEND_GOOD_READ_BEEP_ENABLE,PropertyID.SEND_GOOD_READ_VIBRATE_ENABLE}; int[] ret = mScanManager.getParameterString(keys);		

2.15.resetScannerParameters【Reset scan header Settings】

Definition	public boolean resetScannerParameters();		
Description	Reset the scan head Settings to factory defaults.		
Parameters	name	type	note
Return (boolean)	true : Successful false: Failure		
Reference code	mScanManager.resetScannerParameters();		

3. Examples of common Settings

不同模式下部分属性设置参考表		
mode	function	Reference code

Broadcast mode	Turn sound and vibration on or off	ScanManager mScanManager = new ScanManager();
		// The following Settings are for the scan header output mode is intent mode, so there is a SEND on the set attribute identifierm
		
		ScanManager.setPropertyInts(new int[]{PropertyID.SEND_GOOD_READ_BEEP_ENABLE},new int[]{0});
		// Sound 0 off 1 Short 2 sharp
		
		//Vibrate when scanning is successful
		mScanManager.setPropertyInts(new int[]{PropertyID.SEND_GOOD_READ_VIBRATE_ENABLE},new int[]{0});//Vibrate 0 off 1 on

	Set the action for broadcast mode and the keyString for receiving scan results	new ScanManager().setParameterString(new int[]{PropertyID.WEDGE_INTENT_ACTION_NAME,PropertyID.WEDGE_INTENT_DATA_STRING_TAG},new String[]{"Your Action"," the KeyString you received"});

Keyboard mode	Turn on or off keyboard mode for sound and vibration	ScanManager mScanManager = new ScanManager();
		//In the following Settings, the output mode of the scan header is keyboard mode, so there is no SEND on the identifier of the setting property
		
		mScanManager.setPropertyInts(
		new int[]{PropertyID.GOOD_READ_BEEP_ENABLE},new int[]{0});
		//Sound 0 off 1 Short 2 sharp
		
		//Vibrate when scanning is successful
		mScanManager.setPropertyInts(new int[]{PropertyID.GOOD_READ_VIBRATE_ENABLE},new int[]{0});
		//Vibrate 0 off 1 on

	Keyboard type	//The output mode is keyboard mode new ScanManager().setParameterInts(new int[]{PropertyID.WEDGE_KEYBOARD_TYPE},new int[]{1});

		0: physical keyboard 1: Input software 2: Use physical keyboard only 3: Use the input method only
	Set the action key character	new ScanManager().setParameterInts(new int[]{PropertyID.LABEL_APPEND_ENTER},new int[]{1}); //An optional value for Parameters 0 : None 1 : Carriage return 2 : IME action done 3 : TAB

Common functions reference table	
Function	Reference code

Set output mode	new ScanManager().switchOutputMode(0);//Enable Broadcast mode
	new ScanManager().switchOutputMode(1);//Enable keyboard mode

Enable or disable the application identifier	int [] keyInt = new int[]{PropertyID.LABEL_SEPARATOR_ENABLE};
	int [] valueInt = new int[]{1};//0: off 1: on
	new ScanManager().setPropertyInts(keyInt,valueInt);

The scan code is appended with prefix and suffix. The appended format is 0-6	new ScanManager(). setPropertyInts(new int[]{PropertyID.SEND_LABEL_PREFIX_SUFFIX},new int[]{1});
Replace	new ScanManager() .setParameterString(new int[]{PropertyID.LABEL_MATCHER_TARGETREGEX,PropertyID.LABEL_MATCHER_REPLACEMENT},new String[]{"oldStr","newStr"});

Multiple bar code Settings (may not work on every device)	int[] multiConfigbuf = new int[]{
	PropertyID.MULTI_DECODE_MODE,
	PropertyID.FULL_READ_MODE, PropertyID.BAR_CODES_TO_READ};
	int[] multiConfigval = new int[3];
	multiConfigval[0] = 1;//0 Disable multi-bar code mode. 1 Enable multi-bar code mode
	multiConfigval[1] = 1;//0 Generates a decoding event after one or more barcodes are decoded 1: A callback is generated to the application only if it is decoded to at least the number of set barcodes.
	multiConfigval[2] = 4;//This parameter sets the number of decodes, 1-10 barcodes. The default value is 1.
	new ScanManager().setParameterInts(multiConfigbuf, multiConfigval);
	
	Collected:
	// Broadcast to obtain multi-bar code scan results, you need to set the output mode of the scan head to broadcast output.
	String moreBarcodeStr=intent.getStringExtra("com.ubx.datawedge.data_string");

Deduplication in continuous scan mode	new ScanManager().setParameterInts(new int[]{PropertyID.DEC_Multiple_Decode_MODE},new int[]{0});//0：可重复解码 1：连续不重复解码 2：缓存内不重复解码

Set the interval for scanning two identical bar codes	//value * 100 ms; MIN == 1,MAX == 99 Set the interval for scanning two identical bar codes, for example, transfer 1--> 1*100ms == 100ms; The maximum transmission is 99 --> 99*100ms == 9900ms, that is, the interval is 9900 milliseconds
	mScanManager.setPropertyInts(new int[]{PropertyID.TIMEOUT_BETWEEN_SAME_SYMBOL},new int[]{1});

Set scanning interval and scanning timeout period	ScanManager mScanManager = new ScanManager();
	mScanManager.setPropertyInts(new int[]{PropertyID.DEC_Multiple_Decode_INTERVAL},new int[]{50});//Scan interval, default 50ms (range: 0-5000ms)
	mScanManager.setParameterInts(
	new int[]{PropertyID.DEC_Multiple_Decode_TIMEOUT},new int[]{5000});//Scan timeout period, default 5000ms (Range: 50-60000ms)

Turn on or off the corresponding code system	ScanManager mScanManager = new ScanManager();
	mScanManager.enableSymbology(Symbology.AZTEC,false);
	//AZTEC：Specific code system: false: not decoded true: decoded
	
	mScanManager.enableAllSymbologies(true);//Enable the decoding of all codes. true: enable false: disable

Start aiming	mScanManager.setParameterInts(new int[]{PropertyID.DEC_PICKLIST_AIM_MODE},new int[]{1});

mode	

The default verification level is 1	int[] codeleveindex = new int[]{ PropertyID.LINEAR_CODE_TYPE_SECURITY_LEVEL};
	int[] codelevevalue = new int[]{1};//level 0-3.
	int ret2 = new ScanManager().setParameterInts(codeleveindex, codelevevalue);

Sets the on or off code and check bit	int[] codeChecksumKey = new int[]{ PropertyID.EAN13_ENABLE,//Turn on the EAN13 code switch, 0: off 1: on
	PropertyID.UPCA_ENABLE,//Set the UPCA code switch. 0: off 1: on
	PropertyID.UPCE_ENABLE,//Set the UPCE code switch. 0: off 1: on
	PropertyID.UPCE1_ENABLE,//Set the UPCE1 code switch. 0: off 1: on
	PropertyID.EAN13_SEND_CHECK,//Set the EAN13 code switch. 0: off 1: on
	PropertyID.UPCA_SEND_CHECK,//Set the UPCA check bit switch. 0: off 1: on
	PropertyID.UPCE_SEND_CHECK,//Set the UPCE check bit switch. 0: off 1: on
	PropertyID.UPCE1_SEND_CHECK};//Set the UPCE1 check bit switch. 0: off 1: on
	int[] codeChecksumvalue = new int[]{1,1,1,1,0,0,0,0};
	int ret = new ScanManager().setParameterInts(
	codeChecksumKey, codeChecksumvalue);

Low contrast image mode	ScanManager mScanManager = new ScanManager();
	mScanManager.setPropertyInts(new
	int[]{PropertyID.LOW_CONTRAST_IMPROVED,PropertyID.LOW_CONTRAST_IMPROVED_ALGORITHM} ,new int[]{2,0});
	// Low contrast image mode, the value can be 0-5
	// Enhance low contrast image algorithm 0: off 1: on

Lighting and sight light Settings	new ScanManager().setParameterInts(new int[]{PropertyID.DEC_2D_LIGHTS_MODE},new int[]{1}); //1: Aiming lights only 2: lighting lights only 3: alternating running 4: running simultaneously

Camera frame	//DEC_2D_CENTERING_ENABLE The value is 0 off and 1 on
	//DEC_2D_CENTERING_MODE View mode 0，1，2，3
	mScanManager.setParameterInts(new
	int[]{PropertyID.DEC_2D_CENTERING_ENABLE,PropertyID.DEC_2D_CENTERING_MODE},new int[]{1,2});
	
	//Position coordinate
	int[] key = new int[]{ PropertyID.DEC_2D_WINDOW_UPPER_LX,
	PropertyID.DEC_2D_WINDOW_UPPER_LY,PropertyID.DEC_2D_WINDOW_LOWER_RX,PropertyID.DEC_2D_WINDOW_LOWER_RY };
	int[] value = new int[]{632, 346,640,354};
	mScanManager.setParameterInts(key, value);

Encoding format	int[] key = new int[]{ PropertyID.CODING_FORMAT};
	int[] value = new int[]{1};//0：UTF-8 1：GBK
	mScanManager.setParameterInts(key, value);

OCR decoding mode and OCR template	int[] key = new int[]{ PropertyID.DEC_OCR_MODE,PropertyID.DEC_OCR_TEMPLATE };
	//DEC_OCR_MODE:mode（0-3）DEC_OCR_TEMPLATE：template（0-4）
	int[] value = new int[]{3,0};//3:Both(mode) 0: User Defind (template)
	int ret = mScanManager.setParameterInts(key, value);

OCR 自 Definition template	int[] key = new int[]{ PropertyID.DEC_OCR_USER_TEMPLATE }; String[] action = { "1377777777777777770" }; boolean ret = mScanManager.setParameterString(key, action); The character string begins with 1 The second character represents the font (1, 2, 3,4, 5) and the 3 represents the two fonts ocrA&ocrB The third to the second last represents the character type character (the character represents the character type; The value can be 5 (numeric combination), 6 (letter combination), 7 (numeric combination), or 8. (any supported character combination). The character string ends with 0 For example, an 18-digit ID card:1377777777777777770

Optimized one-dimensional code	ScanManager mScanManager = new ScanManager();
	int[] key = new int[]{ PropertyID.FUZZY_1D_PROCESSING }; //Optimized one-dimensional code
	
	int[] value = new int[]{3 };//0:Off 1: On
	int ret = mScanManager.setParameterInts(key, value);

Exposure Parameters setting	int[] key = new int[]{ PropertyID.IMAGE_EXPOSURE_MODE,PropertyID.DEC_ES_MAX_EXP,PropertyID.DEC_ES_MAX_GAIN, PropertyID.DEC_ES_TARGET_VALUE }; //Exposure Parameters setting
	int[] value = new int[]{1,500,4,4000 };
	int ret = mScanManager.setParameterInts(key, value);

4 Get scan head picture (in broadcast mode)

This method is suitable for obtaining the image decoded after the last laser output of the scanning head. It is recommended to call this method after obtaining the bar code.

4.1 Calling method

Send broadcast:

```
Intent intentImage = new Intent("action.scanner_capture_image");

//intentImage.putExtra("rotate",180);//Rotate picture angle

sendBroadcast(intentImage);
```

4.2 Receive picture data in broadcast mode

To register a picture to receive broadcasts:

```
String action_scanner_capture_image_result = "scanner_capture_image_result";

IntentFilter filter = new IntentFilter();

filter.addAction(action_scanner_capture_image_result);

registerReceiver(mScanReceiver, filter);
```

4.3 Set broadcast callback

```
BroadcastReceiver mScanReceiver = new BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        if(intent != null && intent.getAction().equals(action_scanner_capture_image_result)) {
            byte[] bmpByte = intent.getByteArrayExtra("bitmapBytes");
            if(bmpByte != null && bmpByte.length > 1) {
                Bitmap mBitmap =
```

```
BitmapFactory.decodeByteArray(bmpByte,                              0,
bmpByte.length).copy(Bitmap.Config.RGB_565, true);
            }else {
                Toast.makeText( ScanActivity.this, "Failed to get
image" , Toast.LENGTH_SHORT).show();
            }
        }
    }
};
```

4.4 Unregister

If you no longer want to receive pictures, you can cancel your registration.

```
unregisterReceiver(mScanReceiver);
```

5 Broadcast settings scan Parameters (supported by Android11 and above)

Broadcast settings scan code Parameters ACTION

action	com.android.broadcast.uscanner.settings

Example：

Intent intent = new Intent(“com.android.broadcast.uscanner.settings”);

5.1 Scan engine switch

extra	keyInt
value	int type: -10
extra	valueInt
value	int type 0：close 1：open

Example：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", -10);
intent.putExtra("valueInt", 1);
sendBroadcast(intent);
```

5.2 Start or stop scanning

extra	keyInt
value	int type: -11
extra	valueInt
value	int type 0：Turn off decoding 1：Start decoding

Example：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", -11);
intent.putExtra("valueInt", 1);
sendBroadcast(intent);
```

5.3 Set output mode

extra	keyInt
value	int type: -12
extra	valueInt
value	int type 0: Turn on broadcast mode 1: Keyboard mode 2: Broadcast mode + keyboard mode

Example：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", -12);
intent.putExtra("valueInt", 0);
sendBroadcast(intent);
```

5.4 Trigger mode

extra	keyInt
value	int type: -13
extra	valueInt
value	int type 0：Automatic mode (start after pressing the button, stop after timeout) 1: Continuous scanning mode 2: Manual mode (start after pressing the button, release the button or stop after timeout)

Example：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", -13);
intent.putExtra("valueInt", 0);
sendBroadcast(intent);
```

5.5 Lock scan

extra	keyInt
value	int type: -14
extra	valueInt

value	int type 0：Locked, unable to scan after locked 1: Unlocked, scanned after unlocked

Example：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", -14);
intent.putExtra("valueInt",1);
sendBroadcast(intent);
```

5.6 Barcode broadcast settings

5.6.1 barcode broadcast action

extra	keyInt
value	int type: 200000 (PropertyID.WEDGE_INTENT_ACTION_NAME)
extra	valueStr
value	String type default：“android.intent.ACTION_DECODE_DATA”

Example：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", 200000);
intent.putExtra("valueStr","android.intent.ACTION_DECODE_DATA");
sendBroadcast(intent);
```

5.6.2 Barcode key value key

extra	keyInt
value	int type: 200002 (PropertyID.WEDGE_INTENT_DATA_STRING_TAG)
extra	valueStr
value	String type default：“barcode_string”

Example：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", 200002);
intent.putExtra("valueStr","barcode_string");
sendBroadcast(intent);
```

5.7 Sound and vibration settings in broadcast mode for barcodes

5.7.1 Sound settings

extra	keyInt
value	int type: 6 (PropertyID.SEND_GOOD_READ_BEEP_ENABLE)
extra	valueInt
value	int type 0:No beep 1: short 2: sharp

Example：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", 6);
intent.putExtra("valueInt",1);
sendBroadcast(intent);
```

5.7.2 Vibration settings

extra	keyInt
value	int type: 7 (PropertyID.SEND_GOOD_READ_VIBRATE_ENABLE)
extra	valueInt
value	int type 0: Turn off vibration 1: Turn on vibration

Example：

```
Intent intent = new Intent("com.android.broadcast.uscanner.settings");
intent.putExtra("keyInt", 7);
intent.putExtra("valueInt",1);
sendBroadcast(intent);
```

note: For other Parameters, please refer to the PropertyID class or contact our technical support.